

後輩に ROS 2 を教える羽目になった学生が、採点をやめた話

ros2-cpp-drill — テストが採点するので、採点する人が要らない

nyanziba / github.com/nyanziba/ros2-cpp-drill

何を作ったか

新人に渡す ROS 2 / C++ のドリルです。

rustlings 方式で、穴埋めを埋めるとテストが合否を出します。

```
./drill watch
```

- 全 35 問。読み物 49 章と**章番号まで1対1**
- C++ を先に片付ける構成（`cppb` / `cpp` / ROS 2）
- 課題は gtest / pytest。GUI 要らず

```
ros2-drill — 課題一覧と進捗

$ ./drill list
ROS 2 練習帳

【C++入門編】 docs/cpp-basics/ の講習に対応。ROS 2 は使いません
▶cppb01_declarations 宣言を読む [入門 1章] ←今ここ
  cppb02_scope       スコープと寿命 [入門 2章]
  cppb03_reference    参照で受け取る [入門 3章]
  .....(中略).....

【初級】 講習資料 11~14章
✓01_publisher       トピックに publish する [11章]
  02_subscriber      トピックを購読する [11章]
  03_custom_interface カスタムインターフェースを定義して使う [13章]
  04_service_server   サービスサーバを作る [14章]
  05_service_client   サービスクライアントを作る [14章]

【中級】 講習資料 15~17章
  06_parameters      クラスの中でパラメータを使う [15章]
  07_param_events     パラメータの変更を監視する [15章]
  08_params_yaml      パラメータを YAML で管理する [15章]
  09_launch           launch を Python / XML / YAML で書く [09章 / 15章]
  10_action_server    アクションサーバを作る [16章]
  11_node_test        テストを書く [17章]

【上級】 資料が「別記事」として先送りしている領域
  12_qos              QoS プロファイルを設計する [05章の発展]
  13_executors        Executor とコールバックグループ [14章の発展]
  14_zero_copy        ゼロコピー (unique_ptr とプロセス内通信) [04章の発展]
  15_composition       コンポーネント化する (RCLCPP_COMPONENTS) [04章の発展]

進捗: 1 / 35
[ ] 内が対応する講習資料の章です。./drill read ID でその章を開けます。
```

埋めずに走らせると、こう落ちる

```
未解答のまま走らせる — テストが「次にやること」を言う

$ ./drill run 01
——初級 01_publisher: トピックに publish する ——

ビルド中...
[ RUN      ] DrillTest.topicトピックにpublishしている
~/ros2-cpp-drill/exercises/01_publisher/test/test_exercise.cpp:42: Failure
Value of: drill::spin_until({talker, probe.node}, [&probe]() {return probe.received.size() >= 2;}, 8s)
Actual: false
Expected: true
"topic" に 8 秒待っても 2 件届きませんでした (受信 0 件)。
- create_publisher<std_msgs::msg::String>("topic", 10) を publisher_ に入れましたか？
- create_wall_timer(500ms, ...) を timer_ に入れましたか？
- タイマのコールバックで publisher_->publish(message) を呼んでいますか？

[ FAILED   ] DrillTest.topicトピックにpublishしている (8052 ms)
.....(中略).....
[ PASSED   ] 0 tests.
.....(中略).....

4 FAILED TESTS

× 課題 01_publisher は未完了です
  課題文: ~/ros2-cpp-drill/exercises/01_publisher/README.md
  編集する: ~/ros2-cpp-drill/exercises/01_publisher/src/minimal_publisher.cpp
  ヒント:  ./drill hint 01
```

落ちたテストが「次にやること」を日本語で返す。採点する人が要りません。

間違え方まで拾う（count_++ を ++count_ にした）

```
間違えたとき — 何がどう違うかを日本語で返す

$ ./drill run 01 # count_++ を ++count_ と書いてしまった
[ OK ] DrillTest.topicトピックにpublishしている (1059 ms)
.....(中略).....
~/ros2-cpp-drill/exercises/01_publisher/test/test_exercise.cpp:62: Failure
Expected equality of these values:
  probe.received[1]
    Which is: "Hello, world! 2"
  "Hello, world! 1"
2 通目が "Hello, world! 1" になっていません。count_ を後置インクリメント (count_++) していますか？ 実際の値: "Hello, world! 2"
.....(中略).....
[ FAILED ] DrillTest.本文がHello_worldと連番になっている (1529 ms)
[ PASSED ] 2 tests.
```

4問中2問は通る。落ちた2問が「**後置インクリメントしていますか？**」と聞いてくる。

直して保存すれば、次の課題へ勝手に進む

```
./drill watch — 保存するたびに再テストし、通ったら次の課題へ

$ ./drill watch 01
watch モード (Ctrl-C で終了)
監視中: ~/ros2-cpp-drill/exercises/01_publisher/src/minimal_publisher.cpp
.....(中略).....
——初級 01_publisher: トピックに publish する ——

ビルド中...

.....(中略).....
× 課題 01_publisher は未完了です
.....(中略).....
保存すると自動で再テストします...
.....(中略).....
[ PASSED ] 4 tests.

✓ すべてのテストに合格しました！
  次の課題: ./drill run cppb01 (宣言を読む)

次の課題に移りました: cppb01_declarations
```

`./drill watch` を開いたまま書くだけ。読み物は GitHub Pages にも出しています。

最後に

自分が答えなくていい状態を作る。 作りたかったのはここです。

落ちたテストが次の一手まで言うので、後輩は自分を待たなくて済みます。

書かせるのは公式と同じ字面です。 解き終わると公式ドキュメントが読めます。

github.com/nyanziba/ros2-cpp-drill (MIT)